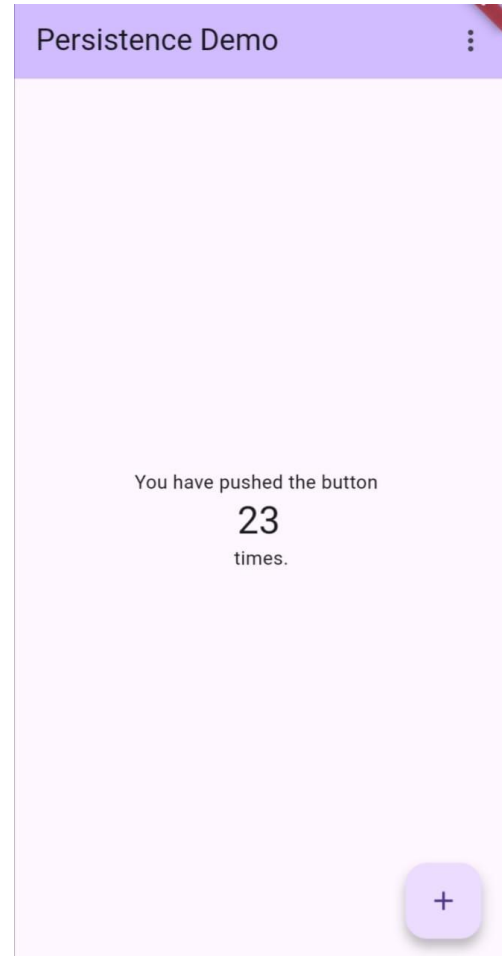


Persistence with Flutter : Lecture 3

IS2205 - Mobile Application Design & Development

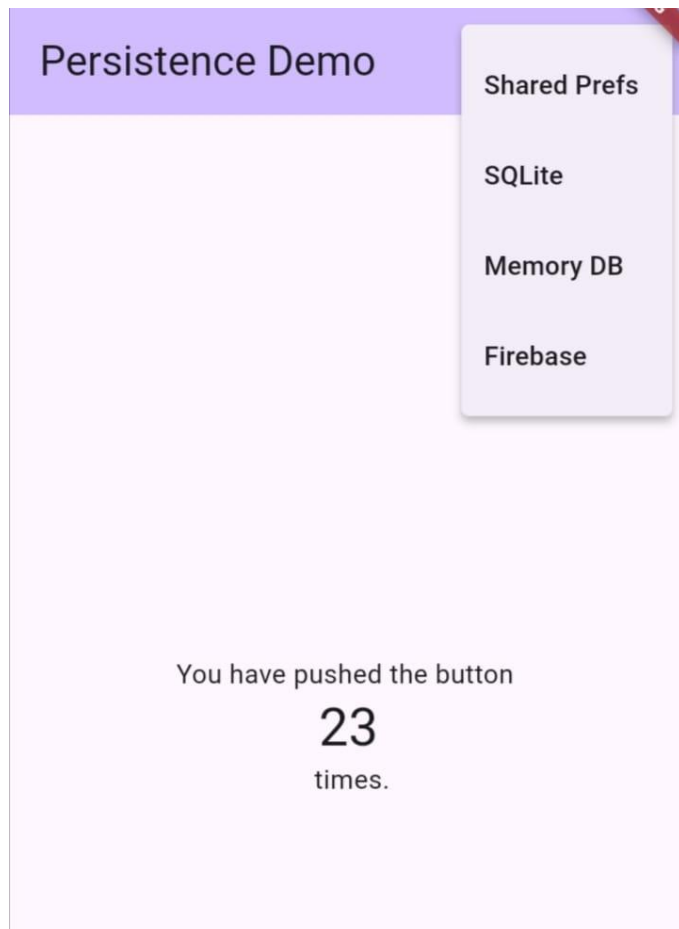
A large, dark blue, curved shape that starts from the bottom left and extends diagonally upwards towards the right, filling the lower half of the slide.

Persistence Demo App



Persistence Demo App

Settings



Default Application

```
void main() => runApp(const MyApp());

// Root widget of the app
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Persistence', // Runtime name
      home: MyHomePage(), // Main screen of the app
    ); // MaterialApp
  }
}
```

Enum effectively
maps code, stored
value and display
name

```
//Enum makes it easier to handle the need  
// rather than defining constants  
enum DataSource {  
    sharedprefs(0, "SharedPreferences"),  
    sqlite(1, "SQLite DB"),  
    hive(2, "In Memory"),  
    firebase(2, "FireBase");  
  
    final int num;  
    final String str;  
    const DataSource(this.num, this.str);  
}
```

Using enum type in menu

```
itemBuilder: (context) => [  
  PopupMenuItem(  
    value: DataSource.sharedprefs.num.toString(),  
    child: Text(DataSource.sharedprefs.str)), // PopupMen  
  PopupMenuItem(  
    value: DataSource.sqlite.num.toString(),  
    child: Text(DataSource.sqlite.str)), // PopupMenuItem  
  PopupMenuItem(  

```

Clean code

StatefulWidget

‘createState’ method
needs to be overridden

After that state is
maintained by ‘State’

```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key});  
  
  @override  
  State<MyHomePage> createState()  
    => _MyHomePageState();  
}
```

The variables used in
our app

initState is called
before drawing the UI

How did Android do it?

```
class _MyHomePageState extends State<MyHomePage> {  
  int _count = 0; // count variable  
  int _dataSource = 0; // Default data source  
  
  // Called when widget is created (before UI build)  
  @override  
  void initState() {  
    super.initState();  
    print('initState()');  
    loadConfig();  
    print("_dataSource $_dataSource");  
    _loadCounter(_dataSource);  
  }  
}
```



Assume :
SharedPreferences are
always available

We save storage option
there permanently

```
// Load config value from SharedPreferences
Future<void> _loadConfig() async {
  final prefs =
    |   await SharedPreferences.getInstance();
  int? i = prefs.getInt('data_source');
  if (i != null && -1 < i &&
    |   i < DataSource.values.length) {
    |   _dataSource = i;
  } else {
    |   _dataSource = 0; //default source
  }
}
```

Dependencies for Persistence Methods

```
dependencies:  
  flutter:  
    sdk: flutter  
  # persistence in shared prefs  
  shared_preferences: ^2.1.1  
  # persistence in sqlite  
  sqflite: ^2.3.0  
  path_provider: ^2.1.2  
  path: ^1.8.3  
  # persistence in memory  
  hive: ^2.2.3  
  hive_flutter: ^1.1.0
```

Pass the data source to the next method

```
class _MyHomePageState extends State<MyHomePage> {  
  int _count = 0; // count variable  
  int _dataSource = 0; // Default data source  
  
  // Called when widget is created (before UI build)  
  @override  
  void initState() {  
    super.initState();  
    print('initState()');  
    _loadConfig();  
    print("_dataSource $_dataSource");  
    _loadCounter(_dataSource);  
  }  
}
```



Note : methods are marked async

Accessing method for DS also awaits

```
Future<void> _loadCounter(int ii) async {  
    if (ii == 0) {  
        // Load stored count from Shared Prefs  
        final prefs = await SharedPreferences.get  
        int? c = prefs.getInt('counter');  
        print("Shared Prefs Loaded : $c");  
        _count = c ?? 0;  
        setState(() => _count);  
    } else if (ii == 1) {  
        int c = await db.CounterDB.getCounter();  
        print("SQLite DB Loaded : $c");  
        _count = c;  
        setState(() => _count);  
    } else if (ii == 2) {
```

Concurrency



Need for Future, async and await

Dart runtime recognizes the methods marked with these keywords

They imply the section needs to be handled with concurrency

Future(class) : Will return in future

async(keyword) : method has to be run asynchronously

await : hold the execution of the particular block until it returns (while making the app responsive)

Building the UI

```
// Build the UI (re-runs every time setState is called)
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Persistence Types'), // Title on app bar
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      actions: [
        // Popup menu button in AppBar

```

Note : methods are marked async

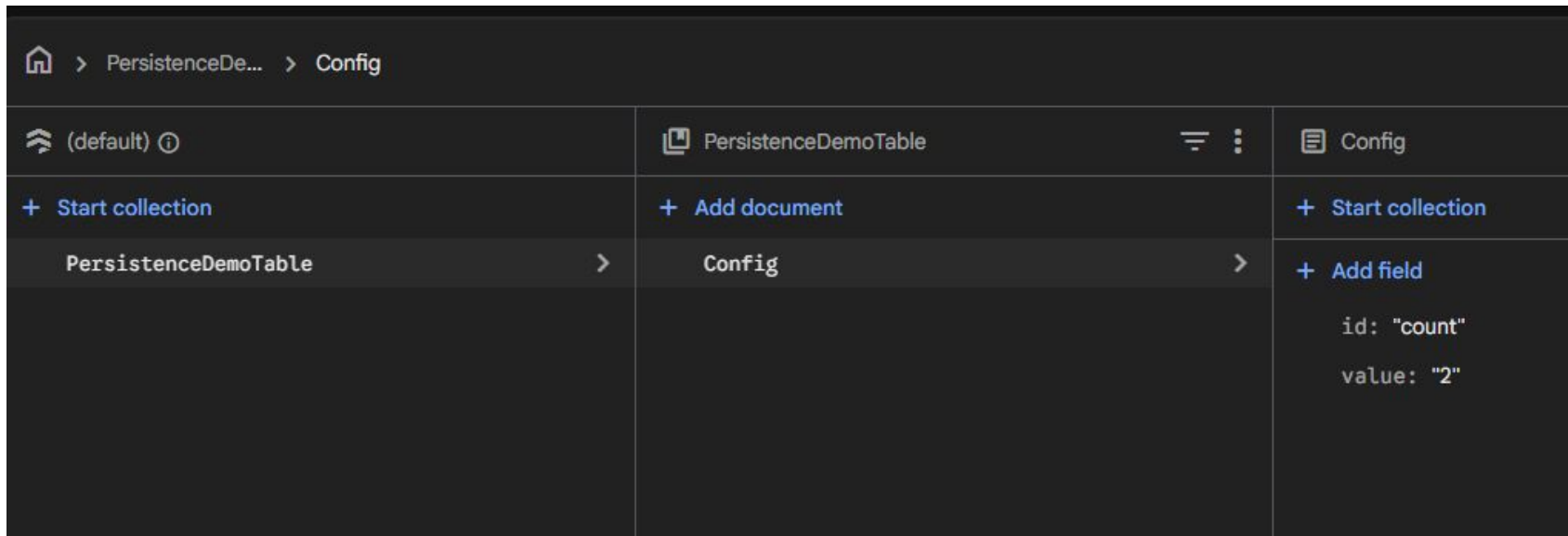
Accessing method for DS also awaits

```
Future<void> _loadCounter(int ii) async {  
    if (ii == 0) {  
        // Load stored count from Shared Prefs  
        final prefs = await SharedPreferences.get  
        int? c = prefs.getInt('counter');  
        print("Shared Prefs Loaded : $c");  
        _count = c ?? 0;  
        setState(() => _count);  
    } else if (ii == 1) {  
        int c = await db.CounterDB.getCounter();  
        print("SQLite DB Loaded : $c");  
        _count = c;  
        setState(() => _count);  
    } else if (ii == 2) {
```

Firebase datastore

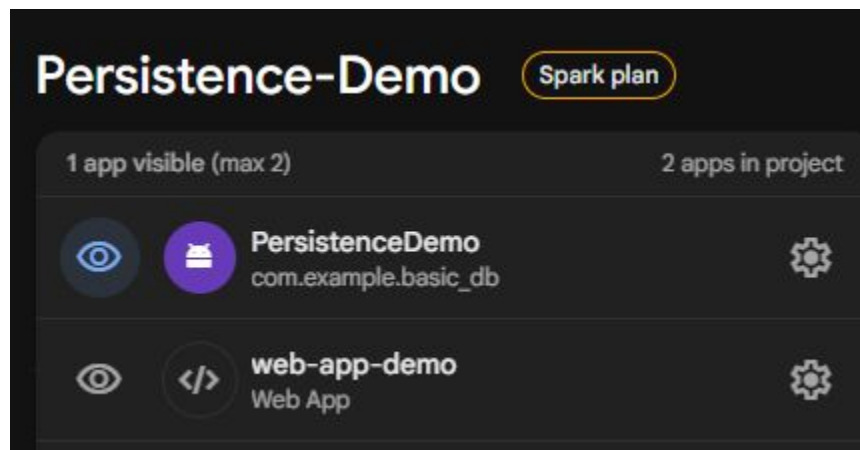


Connecting to firebase



Create in the web console

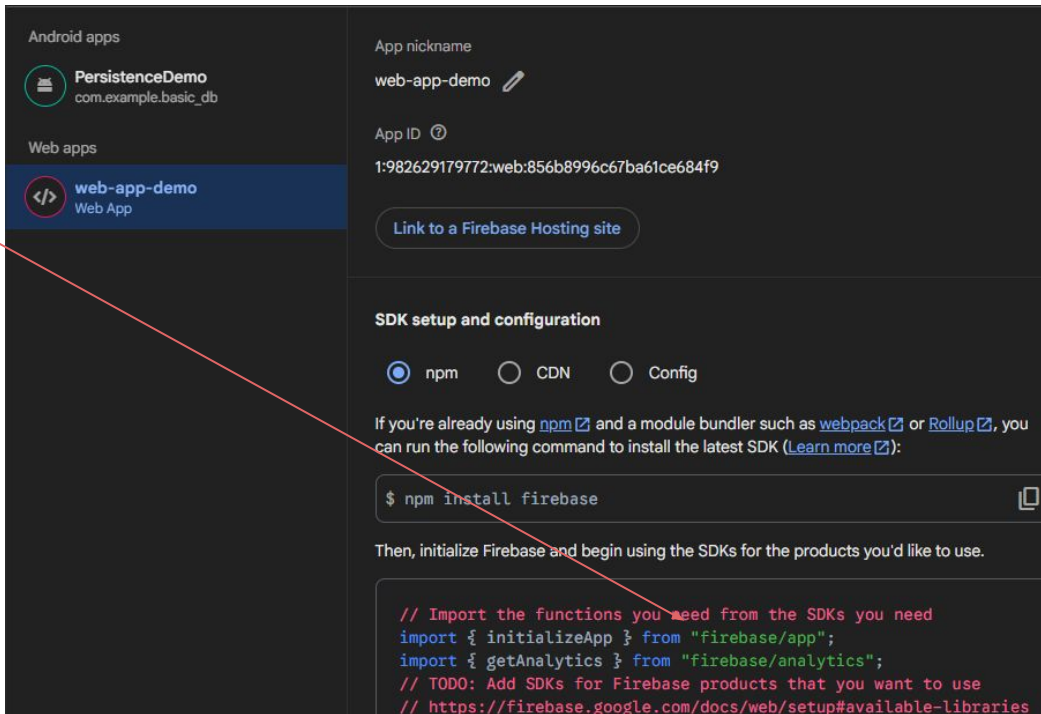
Connecting to firebase...



Apps for the datastore

Connecting to firebase...

Connection Parameters



The screenshot displays the Firebase console interface for a project named 'PersistenceDemo'. On the left sidebar, under 'Web apps', the 'web-app-demo' entry is selected. The main panel shows the configuration for this web app. The 'App nickname' is 'web-app-demo' and the 'App ID' is '1:982629179772:web:856b8996c67ba61ce684f9'. A button labeled 'Link to a Firebase Hosting site' is visible. Below this, the 'SDK setup and configuration' section shows 'npm' as the selected installation method. A text block explains that if using npm and a module bundler like webpack or Rollup, the user can run a command to install the latest SDK. A terminal snippet shows the command '\$ npm install firebase'. Finally, a text block instructs the user to initialize Firebase and use the SDKs for the products they want to use, followed by a code block showing the required imports for Firebase App and Analytics.

Android apps

- PersistenceDemo
com.example.basic_db

Web apps

- web-app-demo**
Web App

App nickname
web-app-demo

App ID ⓘ
1:982629179772:web:856b8996c67ba61ce684f9

[Link to a Firebase Hosting site](#)

SDK setup and configuration

☒ npm ☐ CDN ☐ Config

If you're already using [npm](#) and a module bundler such as [webpack](#) or [Rollup](#), you can run the following command to install the latest SDK ([Learn more](#)):

```
$ npm install firebase
```

Then, initialize Firebase and begin using the SDKs for the products you'd like to use.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries
```

Connecting to firebase...(Android)

Internet connectivity permission : `<uses-permission
android:name="android.permission.INTERNET"/>`

Google services dependencies : `id
"com.google.gms.google-services" version "4.X.Y"`

Google-services.json

Need to reconfigure the Firebase SDKs for your app? Revisit the SDK setup instructions or just download the configuration file containing keys and identifiers for your app.



See SDK instructions



google-services.json

SQLite database



Sqlite with Flutter

```
1  import 'package:sqflite/sqflite.dart';
2  import 'package:path/path.dart'; //for constructing file paths in a platform-independent way
3
4  class CounterDB {
5      // _getDatabase : private method to the class as no need to access from outside
6      static Future<Database> _getDatabase() async {
7          final path = join(await getDatabasesPath(), 'counter.db');
8          // Open or create the database at the specified path
9          return openDatabase(
10             path,
11             //Single Instance per Path: The path parameter in openDatabase acts as a key.
12             //For 'counter.db', only one Database object is maintained,
13             // regardless of how many times _getDatabase is called.
14             version: 1, // Database version for schema upgrades
15             // Called only when the database is created for the first time
16             onCreate: (db, version) async {
17                 await db.execute(
18                     || 'CREATE TABLE config(id INTEGER PRIMARY KEY, value INTEGER)');
19                 // Insert an initial row with id=0 and value=0
20                 await db.insert('config', {'id': 0, 'value': 0});
21             },
22         );
23     }
```

Sqlite with Flutter...

```
25 static Future<int> getCounter() async {
26   final db = await _getDatabase();
27   // Query the 'counter' table for the row where id=0
28   final result = await db.query('config', where: 'id = ?', whereArgs: [0]);
29   // Return the 'value' field from the first row as an integer
30   return result.first['value'] as int;
31 }
32
33 static Future<void> updateCounter(int value) async {
34   // Get the database instance
35   final db = await _getDatabase();
36   // Update the 'value' field for the row where id=0
37   await db.update('config', {'value': value},
38     |   where: 'id = ?', whereArgs: [0]);
39 }
40 }
41
```

Activities to try out:

Include a (-) Button as appropriate and make it work with the rest of the working code.

Run in the Android

The End